

Equity Research Desk

A knowledge guide to every tool and technique used to build the project

This document explains, in plain language, what each piece of the stack is, why it was chosen for this specific project, and how it actually works under the hood. Read it before you start building, and come back to any section while you're implementing the matching phase of the build plan.

Contents

- 1. Multi-Agent Orchestration
 - 1.1 LangGraph
 - 1.2 Supervisor Pattern
 - 1.3 Research Planner Agent
 - 1.4 Agent Memory (short-term and long-term)
- 2. Model Context Protocol (MCP)
- 3. Retrieval and RAG
 - 3.1 Embeddings
 - 3.2 Vector Databases and pgvector
 - 3.3 Chunking Strategies and Parent-Child Retrieval
 - 3.4 BM25 (Sparse / Keyword Search)
 - 3.5 Hybrid Search
 - 3.6 Reciprocal Rank Fusion (RRF)
 - 3.7 Cross-Encoder Reranking
- 4. Prompt Engineering Techniques
 - 4.1 Zero-shot and Few-shot Prompting
 - 4.2 Chain of Thought (CoT)
 - 4.3 Self-Consistency
 - 4.4 ReAct
 - 4.5 Structured Output
- 5. Backend and Infrastructure
 - 5.1 FastAPI (Async)
 - 5.2 Docker and Docker Compose
 - 5.3 PostgreSQL
 - 5.4 Redis (Caching, Rate Limiting, Memory)
 - 5.5 JWT Authentication
 - 5.6 Event-Driven Ingestion (Idempotency, Retries, DLQ)
 - 5.7 Circuit Breaker Pattern
- 6. LLMOps, Evaluation, and Observability
 - 6.1 RAGAS
 - 6.2 Golden Datasets and Regression Testing
 - 6.3 LangSmith and Langfuse
 - 6.4 MLflow
 - 6.5 Cost and Latency Tracking
- 7. Responsible AI and Security
 - 7.1 Prompt Injection Defense

7.2 PII Masking

7.3 Guardrails

7.4 Multi-Dimensional Confidence Scoring

8. CI/CD with GitHub Actions

9. How It All Fits Together

1. Multi-Agent Orchestration

This is the layer that decides which agent runs when, what information passes between them, and when the whole process is finished. Without it, you just have several separate LLM calls with no coordination.

1.1 LangGraph

What it is: A framework for building applications as a graph of nodes, where each node is a step (often an LLM call or a tool call), and edges define what happens next. Built by the LangChain team, designed specifically for stateful, multi-step, multi-agent applications rather than simple one-shot prompt chains.

Why it's in this project: This project needs agents that call each other in a specific order (Planner, then Bull and Bear, then Synthesis), that can loop back for another debate round, and that need to pause and resume without losing progress. LangGraph is built exactly for this. It is also the single most requested orchestration skill in current UK job listings, so using it properly is a direct, defensible line on your CV.

How it works: You define a state object (a typed dictionary or Pydantic model) that flows through the graph. Each node is a Python function that reads the state, does its work, and returns updates to the state. Edges connect nodes, and can be conditional, meaning the graph decides which node to go to next based on the current state. A checkpointing (in this project, Redis) saves the state after every node, so a run can be paused, resumed, or replayed from any point.

Watch out for: LangGraph adds real complexity for a genuinely simple pipeline. If your agents never need to loop, branch, or persist state, a plain function chain is easier and just as valid. Use it because this project's debate structure actually needs it, not because it is the fashionable choice.

1.2 Supervisor Pattern

What it is: A design where one central agent (the Supervisor) does not do the actual research or writing itself, but instead decides which specialist agent should act next, and when the overall task is done. It is a manager, not a worker.

Why it's in this project: Without a Supervisor, you would have to hardcode the exact sequence of agent calls. That works for a fixed pipeline, but this project needs the flexibility to run another debate round if the evidence is weak, or to stop early if the Bull and Bear agents already agree. The Supervisor makes that a runtime decision instead of a hardcoded one.

How it works: The Supervisor reads the current state (what has each agent produced so far, how many rounds have run) and outputs a routing decision: which node to call next, or whether to terminate. A hard ceiling on the number of rounds is built in, so the Supervisor can never let the graph loop forever, which is important both for cost control and because 'how do you stop an agent looping forever' is a real interview question.

1.3 Research Planner Agent

What it is: An agent that runs first, before any research happens, and decides the scope of the investigation: which filings to pull, how many quarters back to look, whether earnings calls or recent news matter more for this particular ticker, and what retrieval settings to use.

Why it's in this project: Without a Planner, the Bull and Bear agents always pull the same fixed set of documents regardless of the company. A mature tech company and a recent IPO need very different research scopes. Adding a Planner is what separates a scripted pipeline (always do steps 1, 2, 3) from a system that reasons about its own research process, which is a much stronger story in an interview.

How it works: The Planner takes the ticker and any basic metadata (sector, whether it recently IPO'd, how volatile the stock has been) and outputs a structured research plan: a list of document types to fetch, a date range, and a priority ordering. The Bull and Bear agents then operate strictly within that plan rather than deciding their own scope.

1.4 Agent Memory (short-term and long-term)

What it is: Two different kinds of memory an agent system can have: short-term memory holds the state of the current run (what has happened so far, within one request), and long-term memory holds knowledge that persists across separate runs (what was learned in previous research sessions).

Why it's in this project: If someone asks about the same ticker twice, the system should recall what it found last time rather than researching from zero every time, the same way a human analyst would refer back to their own notes. This distinction between short-term and long-term memory is a common interview probe on agent design, so being able to explain the difference clearly and point to a working implementation matters.

How it works: Short-term memory is handled by LangGraph's checkpointing, backed by Redis, and only lives for the duration of one research run (though it can pause and resume). Long-term memory works differently: past research notes are embedded and stored in pgvector, tagged with the ticker, so that a new run on the same ticker can retrieve relevant prior findings as additional context before starting fresh research.

2. Model Context Protocol (MCP)

Model Context Protocol

What it is: An open protocol, originally introduced by Anthropic, that standardises how an AI application exposes tools and data sources to a language model. Instead of every application inventing its own way of wiring up 'functions the model can call,' MCP defines a common client-server structure for it.

Why it's in this project: Rather than hardcoding Python functions that the Bull and Bear agents call directly, this project exposes financial data access (fetching filings, transcripts, price history, and news) through a custom MCP server. This is a genuine differentiator: MCP appeared in essentially zero UK job listings in 2024 and grew sharply by 2026, and very few junior candidates can say they have built an MCP server rather than just consumed one someone else built.

How it works: An MCP server declares a set of tools (in this project: `fetch_filing`, `fetch_transcript`, `fetch_price_history`, `fetch_news`), each with a defined input schema and description. An MCP client (your agent's orchestration code) can discover what tools are available and call them using a standard message format, without needing to know the tool's internal implementation. This mirrors how the whole rest of the AI ecosystem is standardising on 'plug in a data source once, use it anywhere.'

Watch out for: MCP is still a young protocol as of 2026, expect the SDKs and conventions to keep shifting. Build against the current stable version and don't over-invest in edge-case features that might change.

3. Retrieval and RAG

RAG stands for Retrieval-Augmented Generation. Instead of relying purely on what the language model memorised during training, the system retrieves relevant real documents at query time and feeds them into the prompt, so answers are grounded in actual, checkable evidence rather than the model's possibly outdated or invented knowledge.

3.1 Embeddings

What it is: A way of converting text into a list of numbers (a vector) such that texts with similar meaning end up with similar vectors. This lets a computer compare meaning mathematically, using distance between vectors, rather than just matching exact words.

Why it's in this project: Every filing, transcript, and news snippet needs to be searchable by meaning, not just by keyword. Embeddings are the foundation that makes semantic search possible at all.

How it works: An embedding model (for example, an OpenAI or open-source sentence embedding model) takes a chunk of text and outputs a fixed-length vector, often several hundred to a few thousand numbers long. Two chunks discussing similar ideas, even with different wording, end up with vectors that are close together in that high-dimensional space, measured using cosine similarity or a similar distance metric.

3.2 Vector Databases and pgvector

What it is: A database purpose-built (or extended) to store embeddings and search them efficiently by similarity, instead of by exact match. pgvector is an extension that adds this capability directly to PostgreSQL.

Why it's in this project: This project chose pgvector specifically: it is one of the two most-cited vector databases in current UK job listings, and using it means you only need to operate one database (Postgres) instead of two separate systems, which is simpler to reason about and explain in an interview. A pure vector database like Qdrant was considered and deliberately rejected for this project, since hybrid search is handled at the application layer anyway, and staying on pgvector also keeps portfolio diversity against another project that already uses Qdrant.

How it works: pgvector adds a new column type to Postgres that stores a vector, and adds index types (such as HNSW) that make nearest-neighbour search fast even across millions of rows. A query like 'find the 10 chunks most similar to this question' becomes a single SQL query using a distance operator, instead of comparing against every row one by one.

Watch out for: Vector search alone is not enough for financial documents, exact figures, ticker symbols, and specific clause numbers are often better found by plain keyword matching, which is why this project also uses BM25 alongside it (see Hybrid Search below).

3.3 Chunking Strategies and Parent-Child Retrieval

What it is: Chunking is how a long document gets split into smaller pieces before being embedded and stored, since embedding an entire filing as one vector loses too much detail to be useful for search.

Parent-child retrieval is a specific chunking strategy that keeps both a small and a large version of each piece of text.

Why it's in this project: Filings and legal-style documents create a real tension: small chunks are precise to search but lack surrounding context, while large chunks have context but dilute the specific detail you were looking for. Parent-child retrieval is the answer this project uses, and it is one of the most frequently asked interview questions in this space.

How it works: Small 'child' chunks (roughly 100 to 150 tokens) are what actually get embedded and searched, because they are precise. Larger 'parent' chunks (roughly 500 to 1000 tokens) contain the surrounding context. When a child chunk is matched by a search, the system fetches its parent chunk and passes that larger, context-rich version to the language model, rather than the tiny child chunk on its own. The short way to remember this: search small, read big.

Watch out for: There is no universal 'correct' chunk size, it depends on the document type and should be benchmarked on your own data, which is exactly why Phase 2 of the build measures a retrieval baseline before tuning anything.

3.4 BM25 (Sparse / Keyword Search)

What it is: A classic text-ranking algorithm that scores how relevant a document is to a query based on keyword overlap, weighted by how rare and how important each term is. It predates embeddings and neural search entirely, but is still extremely effective for certain kinds of queries.

Why it's in this project: Vector search can miss exact terms: a ticker symbol, a specific dollar figure, or an unusual technical term might not have a close semantic neighbour in vector space, but BM25 will find it instantly because it is looking for the literal words. Financial filings are full of exactly this kind of precise, keyword-heavy content.

How it works: BM25 builds an inverted index (a map from each word to every document containing it), then scores documents based on term frequency (how often the query words appear) and inverse document frequency (how rare those words are across the whole corpus), so common words like 'the' contribute almost nothing to the score while rare, specific words contribute a lot.

3.5 Hybrid Search

What it is: Combining both search types, vector (dense, meaning-based) and BM25 (sparse, keyword-based), so that queries benefit from both semantic understanding and exact term matching, rather than picking one and losing the strengths of the other.

Why it's in this project: Pure vector search alone is a common junior mistake, it looks impressive in a demo but underperforms on real queries with exact terms. Explaining when keyword search still wins over vector search is a direct, repeated interview question in your own research.

How it works: Both search methods run against the same corpus, each producing its own ranked list of results. Those two ranked lists then need to be merged into one final ranking, which is the job of Reciprocal Rank Fusion, described next.

3.6 Reciprocal Rank Fusion (RRF)

What it is: A simple, effective method for combining multiple ranked lists of results into a single ranking, without needing the two lists to use comparable scores.

Why it's in this project: BM25 scores and vector similarity scores are not on the same scale and cannot be directly averaged. RRF sidesteps this problem entirely by working with ranks (first place, second place, and

so on) instead of raw scores, which makes it the standard way to fuse hybrid search results.

How it works: For each document, RRF takes its rank position in each of the input lists (for example, position 3 in the BM25 list and position 7 in the vector list) and computes a combined score using the formula $1 / (k + \text{rank})$, summed across all lists, where k is a small constant (commonly 60) that dampens the impact of very low ranks. Documents that rank well in both lists rise to the top of the final result.

3.7 Cross-Encoder Reranking

What it is: A second-pass scoring step that takes the top candidates from an initial search and re-scores them using a model that looks at the query and each document together, rather than comparing precomputed vectors independently.

Why it's in this project: Vector and BM25 search are both fast because they compare precomputed representations, but that speed comes at the cost of accuracy, since the query and document are never actually seen together during scoring. Reranking adds real accuracy back for the handful of candidates that actually matter.

How it works: A cross-encoder model takes a query-document pair as a single combined input and outputs a relevance score directly, rather than comparing two separately-computed embeddings. This is much more accurate but far too slow to run against an entire corpus, so it is only applied to the top 10 to 50 candidates that RRF has already narrowed things down to. This two-stage pattern (fast broad retrieval, then slow precise reranking) is a standard production RAG architecture.

4. Prompt Engineering Techniques

How you phrase and structure the instructions given to the language model has a measurable effect on output quality. This project treats prompting as something to test and measure, not something to guess at.

4.1 Zero-shot and Few-shot Prompting

What it is: Zero-shot means asking the model to perform a task with only an instruction and no examples. Few-shot means including a small number of worked examples in the prompt before asking the model to do the same task on new input.

Why it's in this project: The Bull and Bear agents need to output a consistent, structured thesis every time. Few-shot examples help anchor that structure and the expected level of evidence per claim. The Supervisor's decision, by contrast, is narrow enough that zero-shot with a short rubric is usually enough, and cheaper.

How it works: In a few-shot prompt, you include two or three example input-output pairs directly in the prompt text before the real input, showing the model exactly the format and style you expect. This project treats the choice of zero-shot versus few-shot, and how many examples to include, as something to test against the evaluation dataset rather than assume.

4.2 Chain of Thought (CoT)

What it is: A prompting technique where the model is asked to reason step by step before giving its final answer, rather than jumping straight to a conclusion.

Why it's in this project: The Synthesis agent's job, reconciling two opposing theses into one balanced view, is genuinely reasoning-heavy, and is where an unforced error is most costly. Asking it to think through the reasoning explicitly, rather than just output a verdict, tends to catch inconsistencies that a direct answer would miss.

How it works: The prompt explicitly instructs the model to work through its reasoning first (for example: 'first list where the Bull and Bear theses agree, then list where they conflict and why, then give your final assessment'), and only then produce the structured output. The reasoning trace itself can be logged, which helps with debugging and with explaining a wrong answer later.

4.3 Self-Consistency

What it is: Running the same prompt multiple times (sampling with some randomness) and taking the majority or most common answer, rather than trusting a single generation.

Why it's in this project: A single pass through the Synthesis step could land on a slightly different conclusion each time due to sampling randomness. Self-consistency trades extra cost for more stable, reliable output on the step where reliability matters most.

How it works: The same prompt is sent to the model several times (for example, five times), each generating independently. The final output is chosen by looking at which conclusion or confidence range appears most often across the samples. This project treats the decision to use self-consistency as a measured tradeoff, not a default, since it multiplies token cost by the number of samples for whatever faithfulness gain it produces.

4.4 ReAct

What it is: Short for Reasoning and Acting, a prompting pattern where the model alternates between reasoning about what it needs to do next and actually taking an action (typically calling a tool), rather than planning everything upfront or acting without any reasoning at all.

Why it's in this project: The Bull and Bear agents need to decide which tool to call, look at what comes back, and decide whether they need to call another tool or have enough evidence already. ReAct is the standard pattern for exactly this kind of interleaved reasoning-and-tool-use loop.

How it works: The model's output alternates between a 'thought' (reasoning about the current state and what to do next) and an 'action' (a tool call, such as fetching a filing). After each action, the result is fed back in as an 'observation,' and the model produces another thought. This loop continues until the model decides it has enough information to produce a final answer.

4.5 Structured Output

What it is: Forcing the language model's output into a defined schema (for example, a JSON object with specific fields), rather than accepting free-form prose.

Why it's in this project: Free-text output from an agent cannot be reliably parsed by the next agent in the graph, cannot be evaluated automatically, and cannot be tested for regressions. Every agent in this project returns a structured schema for exactly this reason.

How it works: Using a library like Pydantic, you define the exact shape of the expected output (for example, a thesis object with a list of claims, each claim having a text field and a source citation field). The model is instructed, and often constrained through the API's structured output feature, to return exactly that shape, which can then be validated and used programmatically without fragile text parsing.

5. Backend and Infrastructure

This is the part of the stack that has nothing to do with AI directly, but is what separates a script on your laptop from something that could survive real traffic. It is also the part most portfolio projects skip, which makes it a differentiator.

5.1 FastAPI (Async)

What it is: A modern Python web framework for building APIs, built with native support for asynchronous code and automatic request validation.

Why it's in this project: This project needs an API that clients can call to trigger research runs, and those runs involve waiting on external calls (LLM providers, filing lookups) where async code lets the server handle other requests instead of sitting idle. FastAPI is the standard default for this in the current UK job market.

How it works: Endpoints are defined as Python functions decorated with the HTTP method and path (for example, `@app.post('/research')`). Using `'async def'` instead of `'def'` lets that function pause on a slow operation (like waiting for an LLM response) without blocking the whole server from handling other requests in the meantime. Request and response shapes are defined with Pydantic models, which FastAPI validates automatically.

5.2 Docker and Docker Compose

What it is: Docker packages an application and everything it needs to run (code, dependencies, runtime) into a single, portable container. Docker Compose lets you define and run several containers together as one system, using a single configuration file.

Why it's in this project: This project has several separate pieces that need to run together: the API, a background worker, Postgres, Redis, and the MCP server. Docker Compose lets all of them start with a single command and talk to each other reliably, on your laptop or in the cloud.

How it works: A Dockerfile describes how to build the container for one piece of the app (which base image to use, what to install, how to start it). A `docker-compose.yml` file then lists every container the whole system needs, along with how they connect to each other over an internal network, and running `'docker compose up'` starts everything together.

5.3 PostgreSQL

What it is: A mature, widely used relational (table-based) database.

Why it's in this project: This project uses Postgres as the system of record for everything structured: user accounts, research notes, prompt version history, evaluation results, and cost logs, while the `pgvector` extension on the same database also handles vector search, avoiding the need for a separate system.

How it works: Data is organised into tables with defined columns and types, and relationships between tables (for example, a research note belonging to a user) are expressed with foreign keys. Standard SQL is used to query, join, and filter this data.

5.4 Redis (Caching, Rate Limiting, Memory)

What it is: An extremely fast in-memory data store, commonly used for short-lived or frequently accessed data rather than as a primary database.

Why it's in this project: This project uses Redis for three distinct jobs at once: semantic caching (avoiding repeated LLM calls for similar queries), rate limiting (controlling how often a client can hit the API), and short-term agent memory (LangGraph's checkpoints, so a run can pause and resume).

How it works: Because Redis keeps data in memory rather than on disk, reads and writes are extremely fast, at the cost of being less durable than a full database. For caching, a query is embedded and compared to recently cached queries, if a sufficiently similar one exists, the cached answer is returned instantly instead of calling the LLM again. For rate limiting, a sliding window counter tracks how many requests a given API key has made recently, and rejects requests over the limit with a 429 status and a Retry-After header. For memory, LangGraph writes the graph's state into Redis after every step.

5.5 JWT Authentication

What it is: JSON Web Tokens, a compact, signed format for proving who is making a request to an API, without the server needing to look up a session in a database on every single call.

Why it's in this project: 'How do you secure an enterprise GenAI application' is a direct, recurring interview question. Having real authentication on the API, rather than an open endpoint, is a basic production expectation.

How it works: When a user logs in, the server issues a token containing their identity and an expiry time, signed with a secret key so it cannot be tampered with. On every subsequent request, the client sends that token, and the server verifies the signature to confirm the request is genuine, without needing to store session state itself.

5.6 Event-Driven Ingestion (Idempotency, Retries, DLQ)

What it is: A pattern for processing work asynchronously and reliably: instead of doing everything immediately within one request, the request publishes an event describing the work, a separate worker picks it up later, and failures are handled gracefully rather than silently lost.

Why it's in this project: Ingesting a new filing (fetch, parse, chunk, embed, store) takes real time, and a client should not have to wait for all of that before getting a response. This pattern also handles duplicate uploads and transient failures gracefully, both of which are directly asked about in UK system design interviews.

How it works: A request comes in, the system checks whether this exact document has already been processed (idempotency check), stores the raw file, publishes an event describing the new work, and immediately returns a 202 Accepted response so the client is not left waiting. A background worker consumes that event, does a second idempotency check to guard against duplicate messages, then runs the actual processing. If a step fails for a temporary reason (a network blip), it retries automatically with exponential backoff (waiting longer between each attempt). If it keeps failing after all retries, the event is moved to a dead letter queue for investigation, rather than being silently dropped or crashing the whole pipeline.

5.7 Circuit Breaker Pattern

What it is: A safety mechanism that stops calling an external service once it has failed repeatedly, instead of continuing to hammer something that is clearly down.

Why it's in this project: This project depends on external data sources (SEC filings, transcripts, market data) that can go down or become slow. Without a circuit breaker, a failing external service can cascade into failures across the whole system as requests pile up waiting on it.

How it works: The circuit breaker tracks recent failures for a given external call. After enough failures in a row, it 'opens,' meaning further calls fail immediately without even attempting the network request, giving the failing service time to recover. After a cooldown period, it allows a small number of test requests through, and if those succeed, it 'closes' again and resumes normal operation.

6. LLMOps, Evaluation, and Observability

LLMOps is the practice of running LLM-based systems the way you would run any other piece of production software: measured, monitored, and tested for regressions, rather than shipped once and left alone. This section is described repeatedly in your own research as 'the new system design round' in UK interviews.

6.1 RAGAS

What it is: An evaluation framework specifically built for scoring RAG systems, using metrics designed for exactly this kind of retrieval-plus-generation pipeline.

Why it's in this project: Saying 'it looks good' is not an acceptable answer to 'how do you evaluate an LLM application before shipping,' which is a direct interview question in your research. RAGAS gives concrete, repeatable numbers instead.

How it works: RAGAS scores answers on several dimensions: faithfulness (does the generated answer actually follow from the retrieved evidence, without inventing anything), context precision (how much of the retrieved evidence was actually relevant), and answer relevancy (does the answer actually address the question asked). These scores are usually computed using another LLM as a judge, comparing the generated output against the retrieved context and, where available, a reference answer.

6.2 Golden Datasets and Regression Testing

What it is: A golden dataset is a small, hand-curated set of example inputs with known correct (or expert-labelled) outputs, used as a fixed benchmark. Regression testing means re-running that benchmark automatically whenever something changes, to catch cases where a change quietly makes things worse.

Why it's in this project: Without a golden dataset, any claim about improving the system is just a feeling. This project builds one specifically (20 to 30 hand-labelled ticker theses) so every prompt change, retrieval change, or model change can be measured against a fixed, trustworthy baseline.

How it works: Someone manually reviews and labels the correct or expected output for a fixed set of inputs, and this set is frozen, it does not change when the system changes. Every time you modify a prompt, a chunking strategy, or a model, you re-run the full evaluation against this frozen set and compare the new scores to the previous ones, keeping the change only if the numbers genuinely improve.

6.3 LangSmith and Langfuse

What it is: Both are observability platforms specifically for LLM applications, capturing traces of every step an agent takes: which prompts were sent, what came back, how long each step took, and what it cost.

Why it's in this project: A multi-agent system with several LLM calls per run is very hard to debug by just reading logs. These tools give a visual, step-by-step trace of exactly what happened in a given run, which is essential once the Bull, Bear, Planner, and Synthesis agents are all making their own calls.

How it works: The application is instrumented (often with minimal extra code, sometimes just a few lines of setup) so that every LLM call, tool call, and intermediate step is automatically logged as part of a trace tied to a single request. You can then look at that trace after the fact to see exactly which agent did what, in what order, and how much each step cost in tokens and time.

6.4 MLflow

What it is: An open-source platform for tracking machine learning experiments, including parameters, metrics, and artefacts, so you can compare different runs later.

Why it's in this project: Every prompt experiment run in Phase 6 (which version, what scores it got, what it cost) needs to be logged somewhere comparable, otherwise you cannot properly answer 'how do you know a prompt change actually improved anything.'

How it works: Each experiment run (for example, one prompt version tested against the golden dataset) is logged with its parameters (which prompt, which model), its metrics (RAGAS scores, cost, latency), and any relevant artefacts. MLflow's interface then lets you compare runs side by side to see which combination performed best.

6.5 Cost and Latency Tracking

What it is: Explicitly measuring and logging how much each request costs in tokens (and therefore money) and how long it takes, rather than assuming it is fine.

Why it's in this project: 'Your inference cost is exploding in production, give me five levers to pull' is a direct interview question found repeatedly in your research. You cannot answer it credibly without actually having measured cost and latency in a real system.

How it works: Every LLM call logs the number of input and output tokens used, which is converted into an actual dollar cost based on the provider's pricing, and the time taken is recorded alongside it. These are aggregated per request and per agent, so you can see exactly which agent or step is the most expensive or slowest part of a run, and target caching, batching, or model routing at that specific bottleneck.

7. Responsible AI and Security

Because this project touches financial information and produces output that could be mistaken for advice, the responsible AI layer is not optional polish, it directly reflects the kind of regulatory awareness that UK fintech employers explicitly look for.

7.1 Prompt Injection Defense

What it is: Protecting the system against malicious instructions hidden inside content it retrieves or processes, for example a filing or news article that contains text trying to manipulate the AI into ignoring its original instructions.

Why it's in this project: This project pulls in real external documents (filings, news, transcripts) that the system does not control the content of. Any of that content should be treated as untrusted, since it could contain text specifically crafted to hijack an agent's behaviour.

How it works: Retrieved external text is scanned before being passed into a prompt, looking for patterns that resemble instructions rather than normal document content, and can be flagged or stripped. More fundamentally, the system prompt clearly separates 'instructions from the developer' from 'content retrieved from external sources,' so the model is less likely to treat retrieved text as a command.

7.2 PII Masking

What it is: Detecting and removing or obscuring personally identifiable information (names, addresses, account numbers, and similar) before it is processed or stored.

Why it's in this project: If a user uploads their own document for analysis, or if retrieved content happens to contain personal information, that data should not flow unmasked into logs, caches, or third-party LLM providers.

How it works: A detection step (often a mix of pattern matching and a small classifier model) scans text for likely PII before it is sent onward, replacing detected values with placeholders, so the actual analysis and any logs or caches never contain the real sensitive values.

7.3 Guardrails

What it is: A general term for the checks and limits placed around an LLM system to keep its behaviour within safe, intended bounds, covering things like disallowed topics, output format enforcement, and content moderation.

Why it's in this project: 'Discuss PII masking, prompt injection, jailbreak protection, RBAC, guardrails, and content moderation' appeared directly and repeatedly in your own screenshotted interview questions. Guardrails is the umbrella term covering several of the specific protections in this section together.

How it works: Guardrails are implemented as a layer that sits around the core LLM calls, checking inputs before they reach the model (for prompt injection, disallowed requests) and checking outputs before they reach the user (for structure validity, disallowed content, and in this project, the mandatory disclaimer that the output is not financial advice).

7.4 Multi-Dimensional Confidence Scoring

What it is: Instead of giving a single number for how confident the system is in its conclusion, breaking confidence down into separate, more informative dimensions.

Why it's in this project: A single confidence number hides exactly the information a real analyst would want before trusting a report: is this confident because the evidence is strong, or because the sources happened to agree with each other by coincidence? Separating these out is more honest and more useful.

How it works: The Synthesis agent's output includes separate scores for evidence strength (how much and how good the supporting evidence is), reasoning consistency (how internally coherent the synthesis logic is), source diversity (whether the conclusion rests on one document or many independent ones), and data freshness (how recent the underlying evidence is), combined into an overall weighted confidence score.

8. CI/CD with GitHub Actions

GitHub Actions

What it is: GitHub's built-in automation platform, used to run scripts (tests, linting, builds) automatically whenever code is pushed or a pull request is opened.

Why it's in this project: Every serious production codebase runs automated checks before code is merged. Having this in place, even for a portfolio project, is a baseline production signal reviewers look for, and its absence is one of the 'toy project' red flags identified in your own research.

How it works: A YAML configuration file in the repository defines a workflow: a series of steps (install dependencies, run the linter, run the test suite, build the Docker image) that run automatically on GitHub's servers every time code changes. If any step fails, the pull request is flagged, preventing broken code from being merged silently.

9. How It All Fits Together

It helps to see the full request lifecycle once, with every piece of this document mapped to the moment it actually gets used.

1. Client sends POST /research {ticker}
2. JWT verified -> Section 5.5
3. Redis rate limiter checked -> Section 5.4
4. Redis semantic cache checked -> Section 5.4
(cache hit -> return immediately)
5. Research Planner Agent scopes the task -> Section 1.3
6. Bull Agent: MCP tool calls -> Section 2
-> hybrid retrieval (BM25 + vector + RRF) -> Section 3.5, 3.6
-> cross-encoder rerank -> Section 3.7
-> structured thesis output -> Section 4.5
7. Bear Agent: same process, opposite mandate
8. Supervisor decides: another round, or proceed -> Section 1.2
9. Synthesis Agent reconciles both theses -> Section 4.2, 4.3
-> multi-dimensional confidence score -> Section 7.4
-> guardrail checks on the output -> Section 7.3
10. Long-term memory updated in pgvector -> Section 1.4, 3.2
11. Cache updated, cost and trace logged -> Section 5.4, 6.3, 6.5
12. Response returned to client

Every one of these steps is running inside Docker containers, deployed together via Docker Compose (Section 5.2), tested automatically on every code change through GitHub Actions (Section 8), and measured continuously through RAGAS and the golden dataset (Section 6.1, 6.2) so that any change to prompts, retrieval, or models can be judged on real numbers rather than a feeling that it seems better.

How to use this document: read it once in full before starting Phase 1 of the build, then treat each section as a reference to come back to during the matching build phase. You should be able to explain every concept in this document out loud, in your own words, without looking at your notes, before you consider the project interview-ready.